Proof Path Theory

Certificates, Compression, Discovery Fields, and Mathematical Memory

Komorebi P.

Version 1.0.0

Abstract

A finished proof is a certificate of implication and a compressed projection of a larger mathematical object. The larger object may contain alternative derivations, examples, failed routes, exact computations, boundary cases, provenance, and bridges to later questions. This paper develops a finite Proof Path Theory that keeps these layers distinct. A proof field is a typed acyclic justification hypergraph. Certificates are selected certified subfields that derive declared targets from admissible leaves. In the fixed AND-DAG case, dependency ancestry gives a unique minimal certificate. With alternative justifications, minimum certificate selection is NP-complete by reduction from Set Cover.

Compression is defined relative to task families, producing endpoint, audit, and discovery equivalence, a recoverability preorder, vector-valued loss, and expected recovery debt. A latent bridge graph models declared future reach. Its weighted coverage utility is monotone and submodular, yielding a greedy cardinality guarantee. The framework also treats versioned proof objects, evidence ledgers, correlated review, audit queues, and goal-dependent refinement. The accompanying Python package implements the finite theory, validates proof fields, extracts certificates, computes retention policies, and reproduces the examples. The theory preserves deductive standards while formalizing what compression retains, discards, and makes costly to reconstruct.

Status of the work. The mathematical theorems below concern finite declared proof fields, task families, bridge graphs, covariance matrices, and queue models. Their proofs are given in the text. The computational results are exact finite checks or deterministic numerical examples generated from stated inputs. No simulation is presented as an empirical law of mathematical discovery. Historical priority and comparison with neighboring literatures remain separate questions.

Contents

1 Introduction **4**

2 Claim types and logical boundaries **5**

3 Typed proof fields **6**

4 Certificates and verification closure **7**

4.1 The fixed AND-DAG case 8

5 Alternative derivations and certificate complexity **9**

6 Certified paths as a category **10**

7 Compression relative to a task family **11**

7.1 Pure retention operators 12

8 Endpoint, audit, and discovery equivalence **14**

9 Vector-valued loss **14**

10 Compression debt **15**

11 A proof ecology rather than one ideal form **16**

12 Proof compilers and shared ledgers **16**

13 Latent bridge graphs **17**

13.1 Bridge criticality and dominators 17

14 Retention under a budget **18**

15 Versioned proof objects **20**

16 Evidence ledgers and provenance **21**

17 Correlated review **22**

18 Verification bandwidth and audit queues **24**

19 Recursive proof cubes and expansion depth **25**

20 Worked finite examples **26**

20.1 A fixed AND-DAG 26

20.2 Two certified proofs 26

20.3 Set Cover encoded as certificate selection 27

20.4 Bridge retention 27

20.5 Correlated review and audit capacity 27

21 Reference implementation **27**

21.1 Command-line interface 28

21.2 JSON contract 29
21.3 Tests and build discipline 29

22 What the theory establishes **29**

23 Limitations and claim boundaries **30**

24 Open theorem program **30**

25 Conclusion **31**

A Theorem ledger **32**

B Reproduction **33**

C Canonical proof-field schema excerpt **33**

1 Introduction

Mathematics usually publishes a proof after the proof has been compressed. Definitions have stabilized. Failed attempts have disappeared. A long calculation has become a lemma. A representation change that made the argument visible is replaced by a direct sentence. This is often an improvement. The final certificate is easier to verify and easier to reuse.

Compression can also remove mathematical information that is not needed for the present theorem. An example may explain why a definition was chosen. A failed route may isolate the exact boundary of a method. An alternative proof may carry a bridge to another field. A computation may reveal a stronger statement than the one originally sought. None of these objects is automatically part of the final implication

$$A \vdash B.$$

They may still be part of the mathematical event that produced it.

Proof Path Theory studies this larger object without changing the standard of proof. Its central separation is

$$\boxed{\text{certificate of validity} \quad \neq \quad \text{expanded discovery field.}}$$

The certificate answers whether the conclusion follows. The discovery field records how the conclusion became reachable, what alternatives were considered, what was lost during compression, and which omitted structures may matter later.

The finite theory developed here has six connected layers.

1. A typed proof field records claims, admissible leaves, justifications, evidence, and targets.
2. A certified subfield separates proved or otherwise certified transitions from candidates and interpretations.
3. A certificate is a selected subfield that derives the target.
4. Compression is evaluated relative to declared tasks rather than by length alone.
5. A bridge graph represents possible future reach and makes archival retention an optimization problem.
6. Versioning, review, audit, and refinement describe the life of the proof after the first certificate exists.

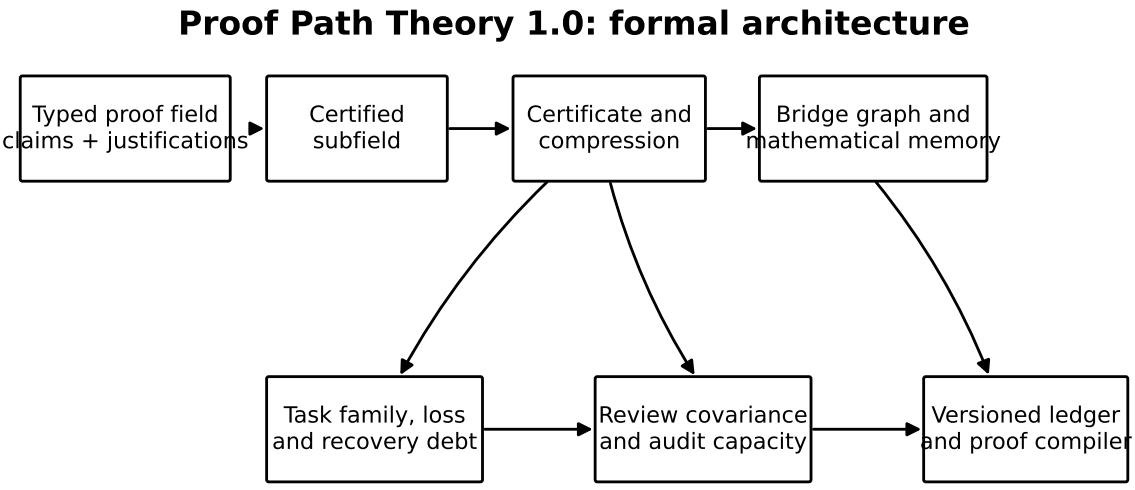


Figure 1: The formal architecture. The certificate is one verified projection of a larger versioned proof field.

The theory is intentionally finite at its foundation. Finite objects make every closure condition and algorithm explicit. Infinite, probabilistic, and higher categorical extensions are listed later as open programs rather than silently assumed.

2 Claim types and logical boundaries

A useful theory of proof paths must not turn every recorded object into a proof step. A notebook may contain a theorem, a conjecture, a failed analogy, a numerical experiment, and a historical interpretation on the same page. Their coexistence does not give them the same logical status.

Definition 2.1 (Evidence kind)**.** An evidence kind is a declared type attached to a record supporting a claim or justification. The core finite vocabulary used here is

formal proof, assumption, exact computation,
numerical evidence, conjecture, external result,
counterexample, interpretation.

An evidence record also states its source, scope, and whether it certifies the justification to which it is attached.

The word “certifies” is deliberately separate from the evidence label. An exact computation may certify a finite identity while providing only numerical evidence for a general statement. An external result may certify a step only on the domain covered by its hypotheses. The scope field is therefore part of the mathematical object.

Warning 2.2 (Evidence types are not one ranking)*.* The evidence kinds do not form a universal total order. A formal proof of a weak statement and an exact computation of a stronger finite instance cannot

be compared by one scalar without specifying the task. The ledger retains type and scope instead of replacing them with a single confidence number.

This separation will be used throughout. Candidate edges may remain in the discovery field, but only certified edges enter a proof certificate.

3 Typed proof fields

The first formal object is a directed hypergraph. A justification may require several premises at once, and a claim may admit several alternative justifications.

Definition 3.1 (Typed proof field)**.** A finite typed proof field is a tuple

$$\mathcal{P} = (V, \mathcal{J}, A, T, \tau, \mathcal{E}, \mathbf{v}, \mathbf{c}),$$

where:

- V is a finite set of claim nodes.
- $\mathcal{J}$ is a finite set of justifications.
- Each $j \in \mathcal{J}$ has a finite premise set $\mathrm{prem}(j) \subseteq V$ and one conclusion $\mathrm{con}(j) \in V$.
- $A \subseteq V$ is the set of required premises of the declared judgment.
- $T \subseteq V$ is the nonempty target set.
- τ assigns node and justification types.
- $\mathcal{E}$ is a typed evidence ledger.
- $\mathbf{v}$ assigns nonnegative value coordinates to nodes.
- $\mathbf{c}$ assigns nonnegative storage, audit, recovery, and cognitive costs.

The dependency graph contains an edge $u \to v$ whenever $u \in \mathrm{prem}(j)$ and $v = \mathrm{con}(j)$ for some justification j. It is required to be acyclic.

A single justification is conjunctive in its premise set. All listed premises must be available. Multiple justifications with the same conclusion are alternatives. This gives the familiar AND/OR structure of proof search without forcing it into an ordinary graph.

Definition 3.2 (Admissible leaf)**.** A node is an admissible leaf when it is a required premise, a declared axiom, or an explicitly cited external result. A zero-premise certified justification may also introduce a leaf when the proof system permits it.

Definition 3.3 (Certified subfield)**.** The certified subfield $\mathcal{P}^{\mathrm{cert}}$ contains every node of $\mathcal{P}$ and only those justifications whose evidence and scope have been declared sufficient to certify the recorded transition.

Definition 3.4 (Derivability)**.** Let $K \subseteq V$ be a selected node set and let $H \subseteq \mathcal{J}$ be a selected justification set. The derivable set $D(K, H)$ is the least subset of K satisfying:

(i) every selected admissible leaf belongs to $D(K, H)$;

(ii) if $j \in H$, $\mathrm{prem}(j) \subseteq D(K, H)$, and $\mathrm{con}(j) \in K$, then $\mathrm{con}(j) \in D(K, H)$.

When only certified justifications are allowed, write $D_{\mathrm{cert}}(K, H)$.

Proposition 3.5 (Finite derivability)**.** *For a finite acyclic proof field, $D(K, H)$ is uniquely determined and can be computed in one topological pass through the dependency graph.*

Proof. Acyclicity gives a topological ordering of the nodes. Begin with the selected admissible leaves. At each later node, test whether one selected incoming justification has all of its premises already derived. Add the node exactly when such a justification exists. No later step can change the status of an earlier node because all dependencies point forward. The resulting set satisfies the closure rules. Any set satisfying the rules must contain every node added by this pass, so the result is the least such set and is unique. □

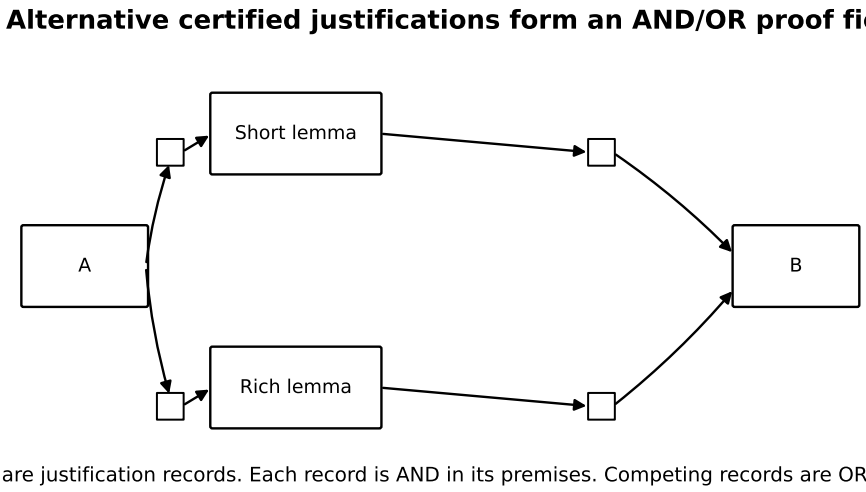


Figure 2: A proof field with alternative certified justifications. Squares represent justification records.

4 Certificates and verification closure

Definition 4.1 (Certificate)**.** A certificate for a requested target set $T_0 \subseteq V$ is a pair

$$\mathcal{C} = (K, H), \qquad K \subseteq V, \qquad H \subseteq \mathcal{J}^{\mathrm{cert}},$$

such that

$$A \cup T_0 \subseteq K \qquad \text{and} \qquad T_0 \subseteq D_{\mathrm{cert}}(K, H).$$

The certificate is node-minimal, justification-minimal, or cost-minimal according to the declared objective.

The canonical target request is usually a subset of the declared set T, although the same object can certify an intermediate node selected for local analysis. This definition preserves the requested endpoint judgment and requires every target to be reconstructed from certified transitions. Extra nodes may remain in a certificate, but they do not help minimality under nonnegative costs.

4.1 The fixed AND-DAG case

The simplest case occurs when every relevant derived node has exactly one certified incoming justification. Then every recorded dependency is mandatory.

Definition 4.2 (AND-DAG)**.** A certified proof field is an AND-DAG relative to target set T_0 when every nonleaf node in the certified ancestry of T_0 has exactly one certified incoming justification.

Let $\mathrm{Anc}(T_0)$ denote the node ancestry obtained by following the unique certified justifications backward from T_0.

Theorem 4.3 (Unique verification core)**.** *Let $\mathcal{P}$ be an AND-DAG relative to T_0. The selected nodes*

$$K_{\mathrm{core}} = A \cup T_0 \cup \mathrm{Anc}(T_0)$$

together with the unique incoming justification of every selected nonleaf node form a certificate. This certificate is the unique node-minimal and justification-minimal certificate for the recorded derivation.

Proof. Backward traversal from each target selects the unique justification that can derive it and every premise required by that justification. Repeating the process terminates because the dependency graph is finite and acyclic. The selected leaves are admissible, and a forward topological pass derives every selected node, so the construction is a certificate.

Let (K, H) be any certificate for the same target set. Each target must be derived. Since it has one certified incoming justification, that justification must lie in H, and all of its premises must lie in K. Applying the same argument recursively forces every node and justification in the backward ancestry into (K, H). Therefore every certificate contains the constructed core. Uniqueness and minimality follow. □

Corollary 4.4 (Linear extraction)**.** *The unique core of an AND-DAG can be extracted in time linear in the size of the relevant certified ancestry.*

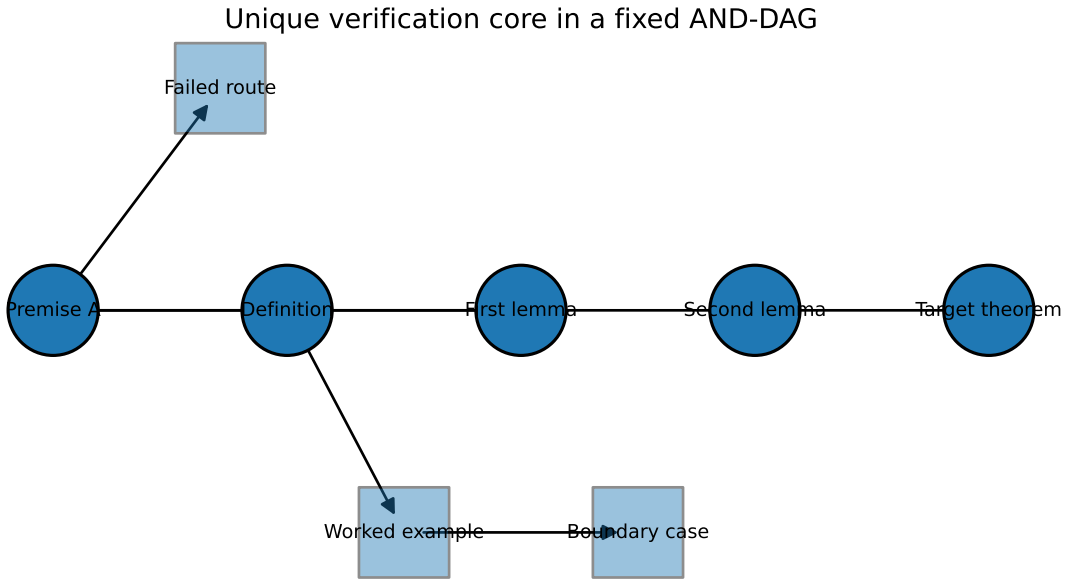


Figure 3: The unique verification core and optional discovery material in a fixed AND-DAG.

Remark 4.5. The theorem identifies the minimum certificate for the recorded derivation. It does not claim that the derivation is globally shortest among all possible proofs of the theorem. A different proof field may have a smaller core.

5 Alternative derivations and certificate complexity

When a claim admits several certified justifications, certificate extraction is no longer a backward closure problem. A local choice may be reused by several later obligations, which couples decisions across the field.

Problem 5.1 (Minimum Certificate Decision)**.** Given a finite acyclic certified proof field, a target set, nonnegative integer node and justification costs, and a budget B, determine whether a certificate of total cost at most B exists.

Theorem 5.2 (Minimum certificate selection is NP-complete)**.** *Minimum Certificate Decision is NP-complete, even when every nonzero cost is a unit node cost and all justification costs are zero.*

Proof. The problem belongs to NP. A proposed node and justification set can be checked in polynomial time by verifying references, computing derivability in topological order, and summing costs.

For NP-hardness, reduce Set Cover. Let $U = \{u_1, \ldots, u_m\}$ be the universe, let $S_1, \ldots, S_n \subseteq U$ be the available sets, and let k be the cover budget. Construct one optional admissible leaf node s_i for each set S_i, with unit cost. Construct one zero-cost obligation node e_j for each element u_j. Whenever $u_j \in S_i$, add a certified justification

$$\{s_i\} \Longrightarrow e_j.$$

Add a zero-cost target node t with the certified justification

$$\{e_1, \ldots, e_m\} \Longrightarrow t.$$

The dependency graph is acyclic. A certificate for t must derive every e_j. Deriving e_j requires at least one selected source node s_i with $u_j \in S_i$. One selected s_i may discharge several obligations, and its cost is paid once. Therefore a certificate of cost at most k exists exactly when the original Set Cover instance has a cover of size at most k. The reduction is polynomial. □

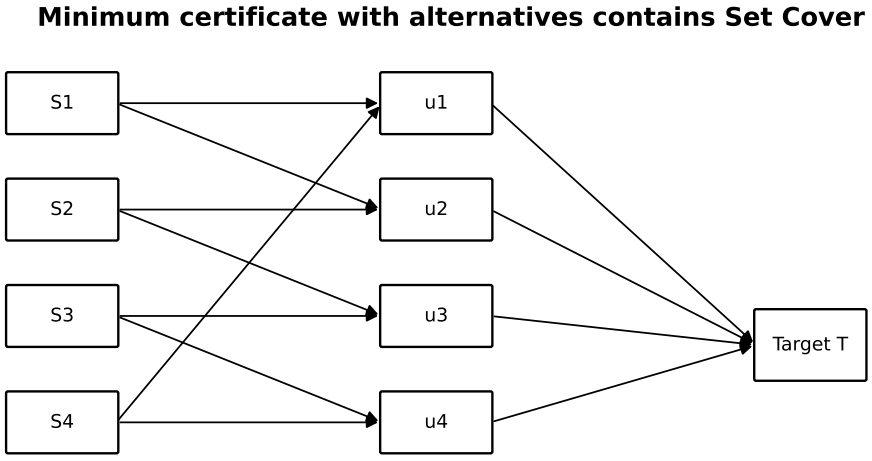


Figure 4: The Set Cover encoding used in the complexity proof.

Remark 5.3 (Algorithmic consequence). Exact minimum certificate search is practical for small fields or fields with few alternatives. Large formal libraries require branch and bound, dynamic programming on restricted structures, integer programming, approximation, or proof-system-specific methods. The reference package includes an exact finite search with an explicit state limit. It does not hide the exponential worst case.

6 Certified paths as a category

The hypergraph model records local justifications. A complementary view treats complete certified transformations as morphisms.

Definition 6.1 (Certified proof category)**.** A certified proof category Cert has mathematical states as objects and certified transformations as morphisms. Identity morphisms retain a state unchanged. Composition is sequential use of compatible certified transformations.

A state may be a proposition, a diagram, an exact finite computation, a coordinate representation, or a structured collection of claims. Candidate analogies may live in a larger discovery graph, but they do not enter Cert until their logical force and domain have been certified.

Proposition 6.2 (Functorial transport)**.** *Let $F : \mathsf{Cert}_1 \to \mathsf{Cert}_2$ be a functor. If*

$$A \xrightarrow{f_1} S_1 \xrightarrow{f_2} \cdots \xrightarrow{f_r} B$$

is a certified path in Cert_1, then

$$F(A) \xrightarrow{F(f_1)} F(S_1) \xrightarrow{F(f_2)} \cdots \xrightarrow{F(f_r)} F(B)$$

is a certified path in Cert_2.

Proof. A functor maps objects to objects and morphisms to morphisms. It preserves identities and composition. The image of the composite path is therefore the composite of the image morphisms, which is certified in the target category. □

The proposition is elementary, but its interpretation is central. A proof may be transported into another language without changing validity. What changes is visibility. A spectral representation may expose a symmetry hidden in physical coordinates. A finite exact computation may expose the correct statement before a general derivation is known. A formal proof assistant may expand dependencies that a journal proof compresses.

Definition 6.3 (Compositional certificate map)**.** A compositional certificate map is a functor

$$\mathcal{C} : \mathsf{Expanded} \longrightarrow \mathsf{Certificate}$$

that sends expanded certified paths to checkable certificate paths and preserves identity and composition.

Proposition 6.4 (Local audit composes)**.** *If every primitive morphism in a finite path has an auditable certificate and the certificate map is compositional, then the composite path is auditable from the primitive certificates and the declared composition rules.*

Proof. Proceed by induction on path length. The length-zero case is an identity. Assume the first r morphisms have an auditable composite certificate and append one certified morphism. Functoriality identifies the new certificate with the composition of the earlier certificate and the primitive certificate of the appended morphism. The declared composition rule completes the audit. □

7 Compression relative to a task family

A representation is not simply good or bad. It is sufficient or insufficient for a declared family of questions.

Definition 7.1 (Task)**.** A task q is a query whose answer is defined on the full proof field. Examples include:

- verify the target theorem;
- reconstruct every primitive dependency;
- teach the conceptual bottleneck;
- recover a boundary example;

- identify all claims affected by a revision;
- search for bridges to a future target family.

A task family is a set $\mathcal{Q}$ of such queries.

Definition 7.2 (Task-sufficient representation)**.** A representation R of a proof field $\mathcal{P}$ is sufficient for task q when there exists a decoder d_q such that

$$q(\mathcal{P}) = d_q(R).$$

It is $\mathcal{Q}$-sufficient when it is sufficient for every $q \in \mathcal{Q}$.

This definition includes exact reconstruction as one task, but it does not require exact reconstruction for every purpose. A certificate can be sufficient for verification and insufficient for teaching. A discovery archive can be sufficient for bridge search and unnecessarily large for routine checking.

Theorem 7.3 (Task information loss criterion)**.** *Let C be a compression map and let q be a task. If there exist proof fields $\mathcal{P}_1$ and $\mathcal{P}_2$ such that*

$$C(\mathcal{P}_1) = C(\mathcal{P}_2) \qquad \textit{but} \qquad q(\mathcal{P}_1) \neq q(\mathcal{P}_2),$$

then the compressed representation is not sufficient for q.

Proof. Suppose a decoder d_q existed. Applying it to the common compressed representation would have to produce both $q(\mathcal{P}_1)$ and $q(\mathcal{P}_2)$. Since these answers differ, no such decoder exists. □

Definition 7.4 (Recoverability preorder)**.** For a fixed task family $\mathcal{Q}$, write

$$R_1 \preceq_{\mathcal{Q}} R_2$$

when every task in $\mathcal{Q}$ that is recoverable from R_1 is also recoverable from R_2.

Proposition 7.5 (Recoverability is a preorder)**.** *The relation $\preceq_{\mathcal{Q}}$ is reflexive and transitive.*

Proof. Every representation supports every task it supports, so the relation is reflexive. If every task supported by R_1 is supported by R_2, and every task supported by R_2 is supported by R_3, then every task supported by R_1 is supported by R_3. Thus the relation is transitive. □

Define $R_1 \sim_{\mathcal{Q}} R_2$ when both $R_1 \preceq_{\mathcal{Q}} R_2$ and $R_2 \preceq_{\mathcal{Q}} R_1$.

Corollary 7.6 (Quotient poset)**.** *The recoverability preorder descends to a partial order on the equivalence classes under $\sim_{\mathcal{Q}}$.*

Proof. Reflexivity and transitivity pass to equivalence classes. Mutual recoverability has been quotiented out, so antisymmetry holds by construction. □

7.1 Pure retention operators

The simplest compression removes nodes without rewriting the retained material. This baseline is useful because it separates information deletion from more complicated summarization or reinterpretation.

Definition 7.7 (Retention projection)**.** Fix a finite node universe V. For a mask $K \subseteq V$, define

$$P_K(S) = S \cap K, \qquad S \subseteq V.$$

Theorem 7.8 (Retention projection algebra)**.** *For masks $K, L \subseteq V$,*

$$P_K \circ P_L = P_{K \cap L} = P_L \circ P_K.$$

Consequently every retention projection is idempotent, and the masks form a Boolean lattice with meet $K \cap L$ and join $K \cup L$.

Proof. For every $S \subseteq V$,

$$P_K(P_L(S)) = (S \cap L) \cap K = S \cap (K \cap L).$$

Intersection is associative and commutative, which gives the composition identity and commutativity. Taking $L = K$ gives idempotence. The power set of V with intersection, union, complement, bottom $\varnothing$, and top V is a Boolean lattice. □

Corollary 7.9 (Task monotonicity under retention)**.** *If $K \subseteq L$, every task whose required nodes and metadata are retained by P_K is also supported by P_L.*

Proof. Every item retained by K is retained by L. Hence every task requirement contained in K is contained in L. □

Pure retention operators commute. Noncommutativity in real proof workflows therefore requires an additional operation such as state-dependent summarization, renaming, quotienting, or reinterpretation. This distinction prevents simple deletion from being credited with structure it does not possess.

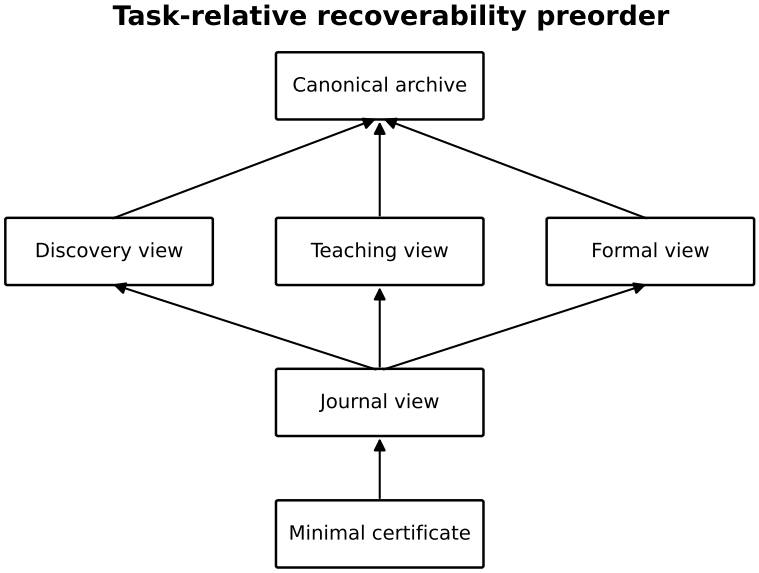


Figure 5: A task-relative recoverability preorder. Teaching, discovery, and formal views may be incomparable.

Theorem 7.10 (Task-family dependence of minimal compression)**.** *There need not be one proper compression that is simultaneously minimal and sufficient for every task family on a proof field.*

Proof. Consider a proof archive containing a certificate and two independent side records x and y. Let q_x ask for x and let q_y ask for y. For the singleton family $\{q_x\}$, a minimal sufficient representation retains the certificate and x but may omit y. For $\{q_y\}$, a minimal sufficient representation may omit x instead. For $\{q_x, q_y\}$, every sufficient representation must distinguish both records. Hence the minimal sufficient representations differ with the task family. The only representation sufficient for every query that can inspect arbitrary archive content is the uncompressed archive itself, which is not a proper compression. □

8 Endpoint, audit, and discovery equivalence

The phrase "the same proof" can refer to several distinct equivalence relations.

Definition 8.1 (Endpoint equivalence)**.** Two representations are endpoint equivalent when they certify the same declared premises and targets.

Definition 8.2 (Audit equivalence)**.** Two representations are audit equivalent when they preserve equivalent dependency, evidence, provenance, and scope information for checking the declared targets.

Definition 8.3 (Discovery equivalence)**.** Two representations are discovery equivalent relative to a declared bridge model when they preserve the same examples, alternatives, boundary cases, candidate routes, and future reach, up to the chosen graph equivalence.

Endpoint equivalence is the weakest of the three. Audit equivalence may fail even when the same theorem is stated, because one document relies on hidden reconstruction or missing provenance. Discovery equivalence is stronger still. Two perfectly valid certificates can expose different mathematical neighborhoods.

Remark 8.4. These equivalences are task-relative. Discovery equivalence requires a declared bridge model. It is not an assertion that every future use of a proof can be predicted.

9 Vector-valued loss

Let a node v carry nonnegative coordinates

$$\mathbf{v}(v) = \big(V(v), D(v), M(v), F(v), A(v)\big),$$

representing verification, discovery, meaning, future bridge, and auditability values. These coordinates are declared model inputs. They may be exact, estimated, or ordinal after a scale has been chosen.

Definition 9.1 (Additive loss vector)**.** For a retained node set $K \subseteq V$, define

$$\mathrm{Loss}(\mathcal{P}, K) = \sum_{v \notin K} \mathbf{v}(v).$$

When normalization is useful, divide each coordinate by the corresponding total coordinate of the full field.

Proposition 9.2 (Zero verification loss can coexist with positive discovery loss)**.** *There exist proof fields and certificates K for which the verification coordinate of* $\mathrm{Loss}(\mathcal{P}, K)$ *is zero while the discovery and bridge coordinates are positive.*

Proof. Take an AND-DAG certificate from A to B and add an optional example node E that is not an ancestor of B. Assign positive discovery and bridge values to E and zero verification value. The unique core omits E while preserving the entire verification ancestry. The verification loss is zero and the other declared losses are positive. □

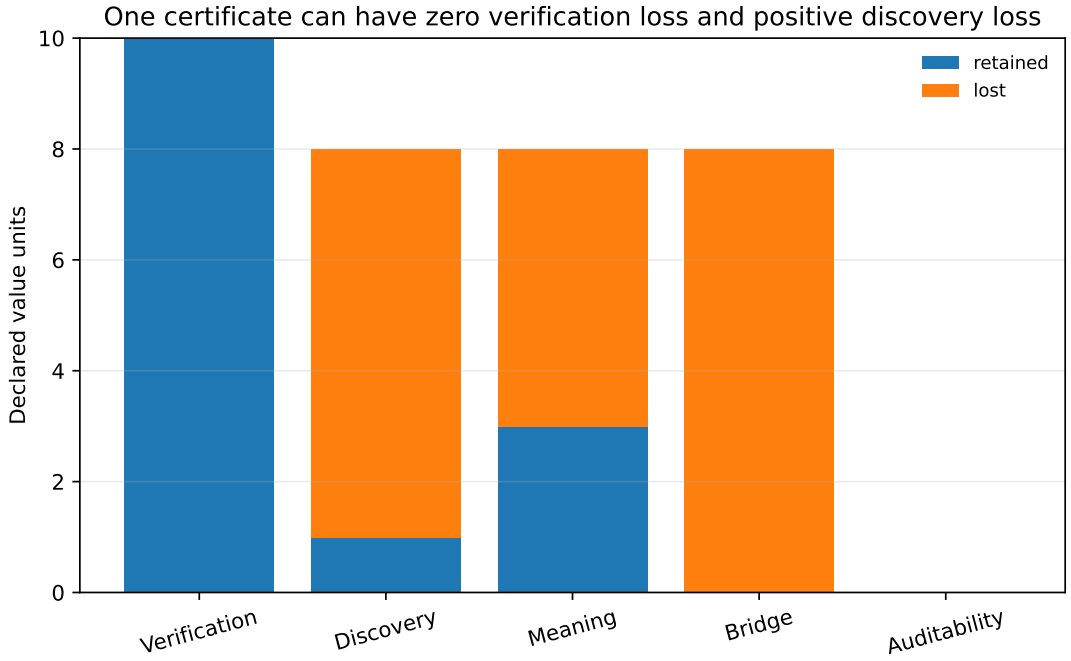


Figure 6: One finite example in which certificate extraction preserves verification and discards discovery value.

No claim is made that all value coordinates are additive. Additivity is a transparent baseline. Interactions can be represented by hyperedge values, submodular utilities, or task-specific decoders when the application requires them.

10 Compression debt

Compression may postpone work rather than eliminate it. A later task may require reconstruction of material that was omitted from the current representation.

Definition 10.1 (Future task model)**.** Let $\mathcal{T}$ be a finite task set with probabilities $p_t \geq 0$ satisfying $\sum_t p_t = 1$. Let $R_t \subseteq V$ be the node set required by task t, and let $c(v) \geq 0$ be the recovery cost of node v when it is absent.

Definition 10.2 (Expected compression debt)**.** The expected compression debt of retained set K is

$$\mathrm{Debt}(K) = \sum_{t \in \mathcal{T}} p_t \sum_{v \in R_t \setminus K} c(v).$$

Proposition 10.3 (Debt is monotone under retention)**.** *If $K_1 \subseteq K_2$, then*

$$\mathrm{Debt}(K_2) \leq \mathrm{Debt}(K_1).$$

Proof. For every task t, the missing set $R_t \setminus K_2$ is contained in $R_t \setminus K_1$. Recovery costs are nonnegative. The taskwise cost cannot increase, and neither can its probability-weighted sum. □

Corollary 10.4 (Irreversible compression accumulates debt)**.** *For a descending sequence of retained sets*

$$K_0 \supseteq K_1 \supseteq \cdots \supseteq K_r,$$

the expected debt is nondecreasing.

A recovery archive can change the cost function. If an omitted node has a stable provenance pointer and a reproducible generator, its recovery cost may be small. If it survives only in memory, its recovery cost may be effectively infinite.

11 A proof ecology rather than one ideal form

The same theorem can support several legitimate representations:

$$\mathcal{R}(B) = \{R_{\mathrm{cert}}, R_{\mathrm{journal}}, R_{\mathrm{teach}}, R_{\mathrm{formal}}, R_{\mathrm{discovery}}, R_{\mathrm{archive}}\}.$$

The certificate is optimized for checking. The journal form balances rigor and readable length. The teaching form expands conceptual bottlenecks. The formal form exposes primitive dependencies. The discovery form retains alternatives and failed routes. The archive preserves reconstruction information.

Definition 11.1 (Representation portfolio)**.** A representation portfolio is a family of proof representations together with stable claim identifiers, translation maps, and a shared status ledger.

Proposition 11.2 (Portfolio robustness)**.** *If several representations lose different information, then their union retains at least as much recoverable information as any one representation alone.*

Proof. Every item recoverable from one representation is recoverable from the union containing it. Additional representations may restore information absent from the first. Recoverable information is therefore monotone under enlargement of the portfolio. □

Redundancy is useful when it is heterogeneous. Storing six copies of the same compressed certificate adds little. Storing a certificate, an expanded explanation, a formal dependency graph, and a reproducible notebook may preserve genuinely different values.

12 Proof compilers and shared ledgers

A proof compiler generates a goal-specific representation from one canonical proof field.

Definition 12.1 (Proof compiler)**.** Let g be a presentation goal and let $\mathcal{P}$ be a versioned proof field. A proof compiler is a map

$$\Gamma_g : \mathcal{P} \longrightarrow R_g$$

that selects, orders, and renders the material required by g while preserving links to canonical claim and justification identifiers.

Typical goals include certificate, journal, teaching, discovery, formal verification, and archive. The compiler does not invent logical status. It reads status from the canonical ledger.

Proposition 12.2 (Shared-ledger consistency)**.** *Suppose every compiled representation is generated from one canonical ledger and every rendered claim retains its ledger identifier. Then a status correction can be propagated consistently to every compiled representation containing that claim by updating the ledger and recompiling.*

Proof. The stable identifier determines every occurrence of the claim across the compiled outputs. Updating the canonical record changes the source value associated with that identifier. Recompilation reads the updated record in every view. A stale status can persist only in an output detached from the canonical ledger or not recompiled. □

This proposition is simple, but it is operationally important. A monograph, a formal library, and a teaching site should not maintain independent truth labels for the same claim.

13 Latent bridge graphs

A proof field records certified dependencies. A latent bridge graph records possible future transfers. The two graphs have different logical roles.

Definition 13.1 (Latent bridge graph)**.** A latent bridge graph is a finite directed graph

$$\mathcal{B} = (W, E_B)$$

whose nodes include retained proof objects, omitted proof objects, and anticipated future targets. An edge $x \to y$ means that x has been declared a possible transfer route toward y. The edge may be certified, conjectural, or exploratory, but its status must be recorded.

Let $S \subseteq W$ be the retained starting set. Let $F \subseteq W$ be a future target set with weights $w_f \geq 0$.

Definition 13.2 (Future reach utility)**.** The weighted future reach of S is

$$U(S) = \sum_{f \in F} w_f \, \mathbf{1}\{f \in \mathrm{Reach}_{\mathcal{B}}(S)\}.$$

This quantity measures declared reach, not truth. A conjectural edge can increase modeled reach without certifying the target. The ledger must keep those statuses separate.

13.1 Bridge criticality and dominators

Definition 13.3 (Bridge-critical node)**.** A node v is bridge critical for target f relative to starts S when f is reachable from S in $\mathcal{B}$ and becomes unreachable after v is removed.

In graph language, v dominates f relative to S when every directed path from S to f passes through v.

Theorem 13.4 (Criticality equals domination)**.** *Assume f is reachable from S. A node $v \notin S$ is bridge critical for f if and only if v dominates f relative to S.*

Proof. If v dominates f, every path from S to f uses v. Removing v removes every such path, so f becomes unreachable. Conversely, if removing v makes f unreachable, then there could not have been a path from S to f avoiding v. Therefore every path used v, which is exactly domination. □

Corollary 13.5 (Weighted criticality)**.** *The utility lost by removing v is the total weight of future targets dominated by v:*

$$U(S) - U_{\mathcal{B}\setminus\{v\}}(S) = \sum_{f \in F:\, v \text{ dominates } f} w_f.$$

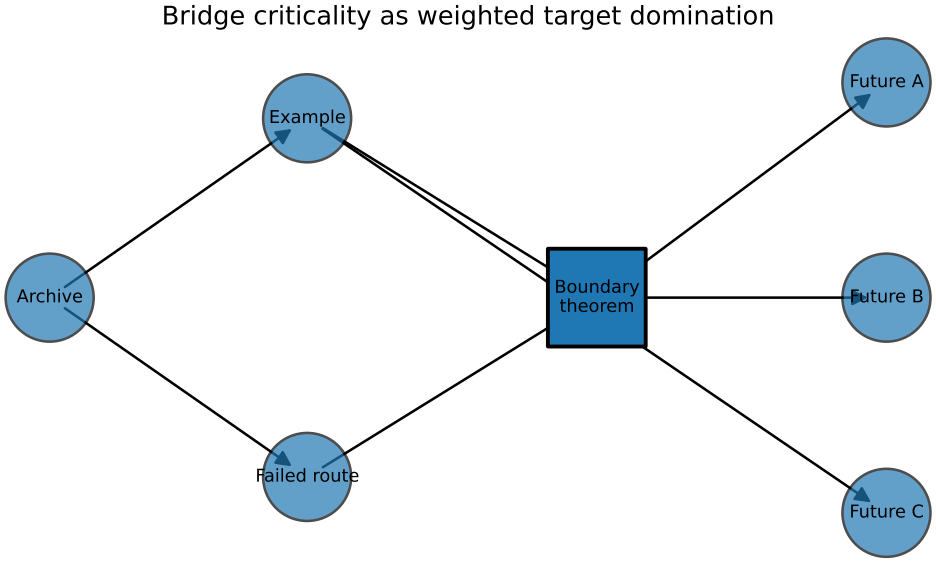


Figure 7: A bridge-critical node dominates two future targets.

A node may have low verification value for the current theorem and high domination value for future targets. This is the precise form of the claim that a forgotten step can remain mathematically important after it has lost its role in the original certificate.

14 Retention under a budget

An archive cannot retain every object at full resolution. The first mathematically controlled retention problem is to choose a finite supplement that preserves as much future reach as possible.

Let $C \subseteq W$ be the candidate nodes that may be retained and let M be a mandatory base set. For $S \subseteq C$, define

$$U_M(S) = U(M \cup S).$$

Theorem 14.1 (Bridge coverage is monotone and submodular)**.** *The future reach utility U_M is a nonnegative monotone submodular set function on C.*

Proof. For each candidate $x \in C$, let $R_x \subseteq F$ be the set of future targets reachable from $M \cup \{x\}$. Let R_M be the targets already reachable from M. Reachability from a union of starting nodes is the union of their individual reach sets, so

$$U_M(S) = \sum_{f \in R_M \cup \bigcup_{x \in S} R_x} w_f.$$

Adding a candidate cannot remove a reachable target, so U_M is monotone.

Now let $S \subseteq T \subseteq C$ and let $x \in C \setminus T$. The marginal gain of adding x is the total weight of targets in R_x that are not already covered. Since T covers at least every target covered by S, the set newly covered from T is contained in the set newly covered from S. Therefore

$$U_M(S \cup \{x\}) - U_M(S) \geq U_M(T \cup \{x\}) - U_M(T),$$

which is submodularity. □

Theorem 14.2 (Greedy cardinality guarantee)**.** *Let S_k be the set obtained by starting from the empty supplement and repeatedly adding the candidate with largest marginal future reach until k candidates have been selected. Let O_k be an optimal supplement of size at most k. Then*

$$U_M(S_k) - U_M(\varnothing) \geq \left(1 - \left(1 - \frac{1}{k}\right)^k\right) (U_M(O_k) - U_M(\varnothing)),$$

and therefore the greedy gain is at least $(1 - 1/e)$ times the optimal gain.

Proof. Write $G_i = U_M(S_i) - U_M(\varnothing)$ and $G^* = U_M(O_k) - U_M(\varnothing)$. By monotonicity and submodularity, the total marginal contribution of the at most k elements of O_k beyond S_i is at least $G^* - G_i$. Hence at least one candidate has marginal gain at least $(G^* - G_i)/k$. Greedy chooses a candidate with at least that gain, so

$$G^* - G_{i+1} \leq \left(1 - \frac{1}{k}\right) (G^* - G_i).$$

Iterating from $G_0 = 0$ gives

$$G^* - G_k \leq \left(1 - \frac{1}{k}\right)^k G^*.$$

Rearranging gives the first bound. The inequality $(1 - 1/k)^k \leq e^{-1}$ gives the second. □

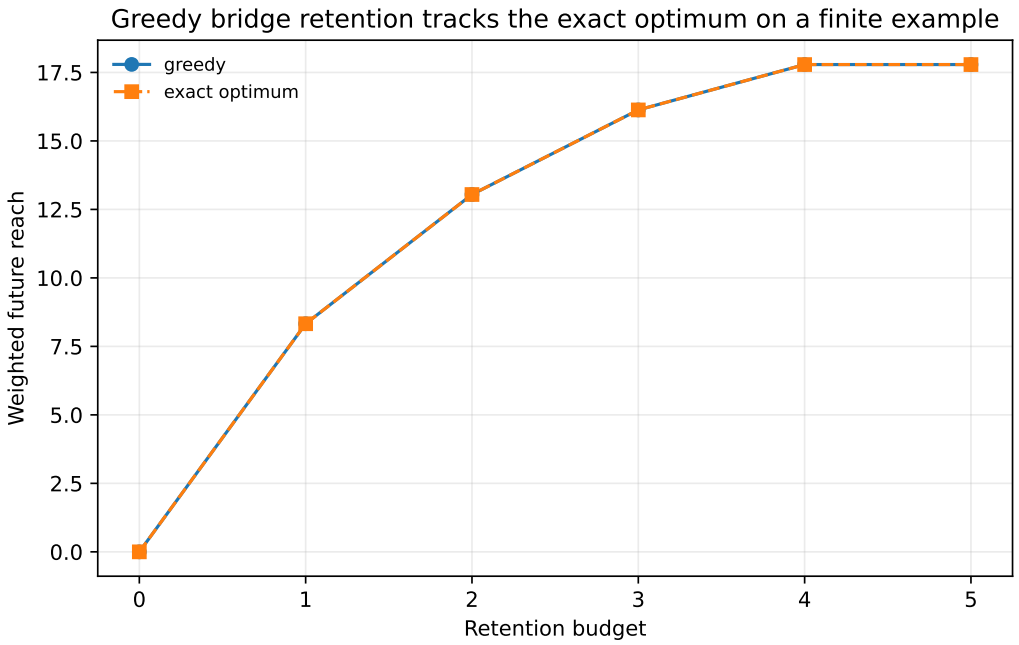


Figure 8: Greedy and exact bridge retention on one reproducible finite example. The theorem gives a worst-case guarantee. The displayed instance performs better.

Warning 14.3 (Arbitrary costs). The simple marginal-gain-per-cost rule included in the software is an interpretable heuristic for arbitrary real costs. The cardinality guarantee above does not automatically apply to that heuristic. Stronger knapsack guarantees require additional algorithmic machinery.

15 Versioned proof objects

A proof archive changes. Claims are strengthened, corrected, refuted, or split. A reproducible theory therefore needs stable identifiers and explicit version relations.

Definition 15.1 (Versioned proof object)**.** A versioned proof object is a sequence

$$\mathcal{P}_0 \longrightarrow \mathcal{P}_1 \longrightarrow \cdots$$

whose nodes and justifications carry stable identifiers. Each transition records additions, removals, altered scopes, evidence changes, and a digest of its parent version.

Definition 15.2 (Append-only certified update)**.** An update $\mathcal{P}_i \to \mathcal{P}_{i+1}$ is append-only certified when every node and justification of $\mathcal{P}_i$ remains unchanged, and $\mathcal{P}_{i+1}$ only adds nodes, evidence, or certified justifications.

Theorem 15.3 (Derivability is monotone under append-only updates)**.** *Every target derivable in $\mathcal{P}_i$ remains derivable in $\mathcal{P}_{i+1}$ under an append-only certified update.*

Proof. A certificate in $\mathcal{P}_i$ uses only nodes and certified justifications that remain unchanged in $\mathcal{P}_{i+1}$. The same selected subfield therefore satisfies the same derivability closure in the later version. □

Retractions and semantic changes are not append-only. They require invalidation analysis.

Definition 15.4 (Syntactic affected set)**.** For a changed node set $X \subseteq V$, define

$$\mathrm{Aff}(X) = X \cup \mathrm{Desc}(X),$$

where descendants are computed in the declared dependency graph.

Theorem 15.5 (Minimal closed syntactic review set)**.** *The set* $\mathrm{Aff}(X)$ *is the unique smallest node set* Y *such that* $X \subseteq Y$ *and* Y *is closed under outgoing dependency edges: whenever* $u \in Y$ *and* $u \to v$ *is a declared dependency edge, then* $v \in Y$*.*

Proof. By construction, every node reachable from X lies in $\mathrm{Aff}(X)$, so the set contains X and is closed under outgoing edges. Let Y be any other set with those two properties. Every directed path beginning in X remains inside Y by induction on path length. Hence every descendant of X lies in Y, and $\mathrm{Aff}(X) \subseteq Y$. □

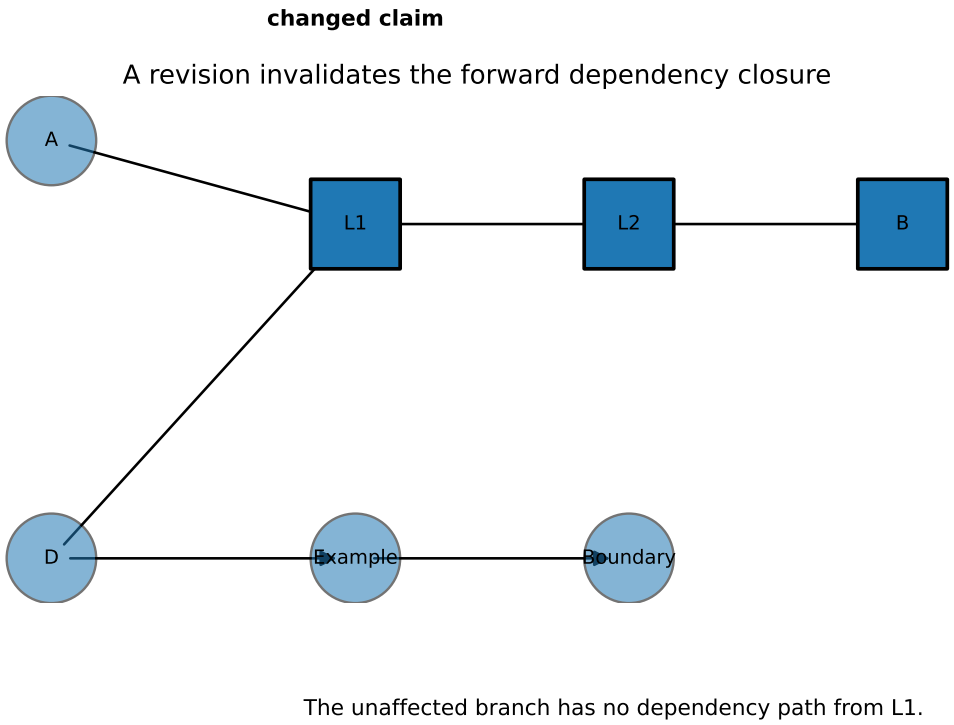


Figure 9: A changed claim and its forward dependency closure.

The theorem is intentionally syntactic. A descendant can survive after rechecking. The affected set says where review is required, not where falsehood is guaranteed.

16 Evidence ledgers and provenance

A claim ledger should answer five questions without reconstructing the entire research history.

1. What is the exact claim identifier?
2. What is its current status and scope?

3. Which justifications depend on it?
4. Which evidence records support or challenge it?
5. Which versions and compiled documents contain it?

Definition 16.1 (Evidence ledger)**.** An evidence ledger is an appendable collection of typed, scoped records linked to claim and justification identifiers. A record may certify, support, challenge, interpret, or refute an object. Its role is explicit rather than inferred from prose placement.

Definition 16.2 (Provenance digest)**.** A provenance digest is a cryptographic hash of a canonical serialization of the claim identifier, content, scope, evidence references, parent version, and relevant build inputs.

A digest does not prove a theorem. It detects unrecorded changes and gives independently generated representations a common reference point.

Proposition 16.3 (Digest stability)**.** *A deterministic canonical serialization gives the same provenance digest for identical records and, except for hash collisions, a different digest after any serialized field changes.*

Proof. Deterministic serialization produces the same byte sequence for identical records, so the hash is identical. Any change that alters the canonical byte sequence changes the hash input. A collision-resistant hash is designed to make equal hashes for distinct byte sequences computationally negligible, though not logically impossible. □

17 Correlated review

Several reviews are not equivalent to several independent reviews. Reviewers may share training, notation, examples, software, or an unnoticed premise.

Let $X_i = \theta + \varepsilon_i$ be reviewer estimates of a quantity θ, with common variance σ^2 and common pairwise correlation ρ. For $n > 1$, assume $-1/(n-1) < \rho \leq 1$, so the mean has positive variance. The mean has variance

$$\operatorname{Var}(\bar{X}) = \sigma^2 \left(\frac{1-\rho}{n} + \rho \right).$$

Theorem 17.1 (Effective reviewer count)**.** *Under the equicorrelated model, the reviewer mean has the same variance as the mean of*

$$\boxed{n_{\text{eff}} = \frac{n}{1 + (n-1)\rho}}$$

independent reviewers of variance σ^2.

Proof. The mean of m independent reviewers of variance σ^2 has variance σ^2/m. Equating this with the correlated mean variance gives

$$\frac{\sigma^2}{m} = \sigma^2 \left(\frac{1-\rho}{n} + \rho \right) = \sigma^2 \frac{1 + (n-1)\rho}{n}.$$

Solving for m gives the formula. □

For fixed positive ρ, n_{eff} approaches $1/\rho$ as n grows. Headcount continues to help, but shared dependence creates a saturation floor.

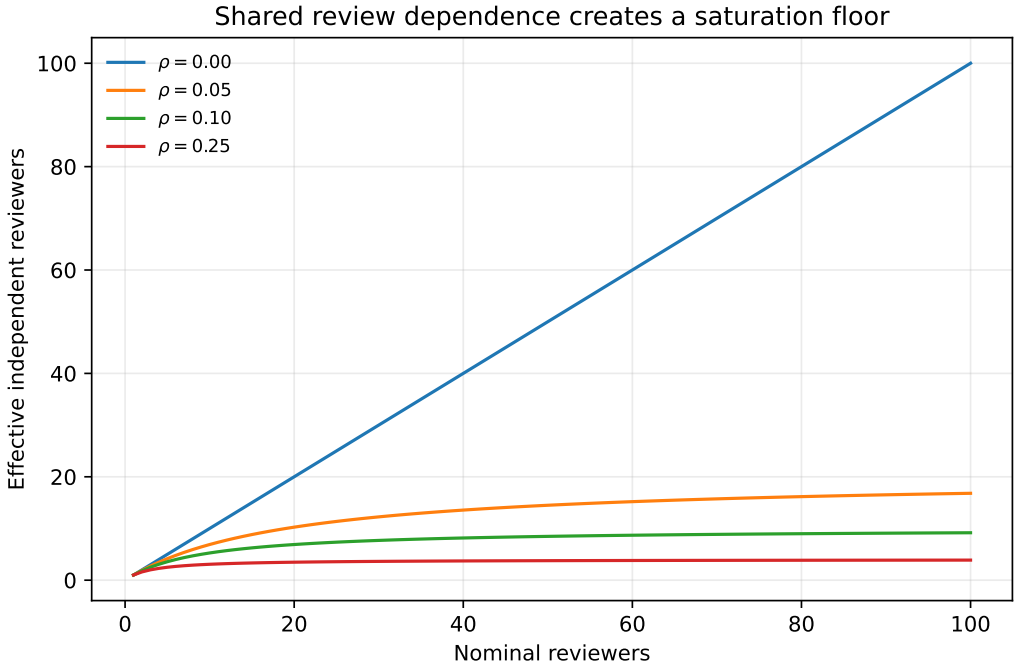


Figure 10: Effective independent reviewer count under equicorrelated errors.

For a general positive definite covariance matrix Σ, consider unbiased linear aggregates $w^\top X$ with $\mathbf{1}^\top w = 1$.

Theorem 17.2 (Minimum-variance covariance-aware weights)**.** *The unique minimum-variance unbiased linear aggregate has weights*

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top \Sigma^{-1}\mathbf{1}}.$$

Its variance is

$$\frac{1}{\mathbf{1}^\top \Sigma^{-1}\mathbf{1}}.$$

Proof. Minimize $w^\top \Sigma w$ subject to $\mathbf{1}^\top w = 1$. The Lagrangian is

$$L(w, \lambda) = w^\top \Sigma w - 2\lambda(\mathbf{1}^\top w - 1).$$

Stationarity gives $\Sigma w = \lambda \mathbf{1}$, hence $w = \lambda \Sigma^{-1}\mathbf{1}$. The constraint gives

$$\lambda = (\mathbf{1}^\top \Sigma^{-1}\mathbf{1})^{-1}.$$

Positive definiteness makes the objective strictly convex, so the solution is unique. Substitution gives the variance formula. □

Warning 17.3 (Bias is not covariance)*.* Covariance-aware weighting does not remove a common bias that is absent from the covariance model. If every reviewer shares bias b, the mean squared error contains the irreducible term b^2.

The practical conclusion is not that consensus is useless. It is that review portfolios should diversify dependence structure, not only increase nominal count.

18 Verification bandwidth and audit queues

A system can generate mathematical candidates faster than it can verify them. Let a_t be arrivals during audit cycle t, let s_t be service capacity, and let B_t be the unresolved backlog. The basic deterministic queue is

$$B_{t+1} = \max\{0, B_t + a_t - s_t\}.$$

Theorem 18.1 (Constant-rate audit regimes)**.** *Suppose $a_t \equiv \lambda$ and $s_t \equiv \mu$.*

(i) *If $\lambda < \mu$, every finite initial backlog reaches zero after finitely many cycles.*

(ii) *If $\lambda = \mu$, the backlog remains constant.*

(iii) *If $\lambda > \mu$, the backlog grows linearly with slope $\lambda - \mu$.*

Proof. Before the truncation at zero, each cycle adds $\lambda - \mu$. When this quantity is negative, the backlog decreases by a fixed amount until it reaches zero. When it is zero, the backlog is unchanged. When it is positive, the truncation is never active after a nonnegative start, and the backlog is $B_0 + t(\lambda - \mu)$. □

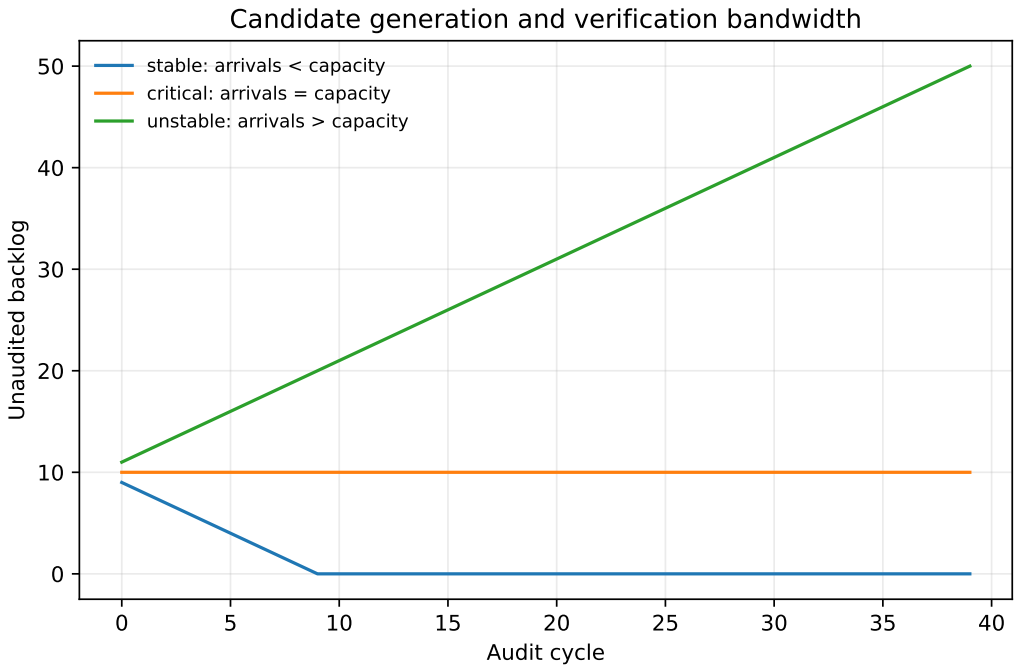


Figure 11: Stable, critical, and unstable audit regimes in the constant-rate model.

A simple audit priority score is

$$P(x) = \frac{p_{\mathrm{err}}(x)\, c_{\mathrm{consequence}}(x)\big(1 + r_{\mathrm{dependency}}(x)\big)}{c_{\mathrm{audit}}(x)}.$$

This is a decision model, not a truth measure. It makes the assumptions behind triage explicit.

19 Recursive proof cubes and expansion depth

A visible proof step is rarely atomic. A lemma opens into definitions and sublemmas. A computation opens into algorithms, parameters, and exact checks. A citation opens into another dependency graph. Proof Path Theory treats this as recursive refinement.

Definition 19.1 (Proof packet and refinement)**.** A proof packet is a node presented at one resolution. A refinement replaces that node by a finite subfield whose certified target reconstructs the original node on the declared scope.

Repeated refinement gives a nested sequence of representations

$$R_0 \prec R_1 \prec \cdots \prec R_d,$$

where deeper levels expose more internal structure. Let $r_d \geq 0$ be the unresolved residual at depth d and let $c_d \geq 0$ be the cumulative cost.

Definition 19.2 (Goal-relative depth objective)**.** For goal g with weights $\alpha_g, \beta_g \geq 0$, define

$$J_g(d) = \alpha_g r_d + \beta_g c_d.$$

An optimal expansion depth is any minimizer of J_g.

Theorem 19.3 (Expansion depth is goal dependent)**.** *The same proof object can have different optimal expansion depths for different goals.*

Proof. It is enough to construct a proof object with exactly two available depths. Choose $d_1 < d_2$ with $r_{d_2} < r_{d_1}$ and $c_{d_2} > c_{d_1}$. Goal g prefers d_2 over d_1 exactly when

$$\frac{\alpha_g}{\beta_g} > \frac{c_{d_2} - c_{d_1}}{r_{d_1} - r_{d_2}},$$

with the usual interpretation when $\beta_g = 0$. Choose one goal ratio below this threshold and another above it. The first prefers the shallower depth and the second prefers the deeper depth. □

A practical stopping rule selects the first depth satisfying either

$$r_d \leq \varepsilon$$

or

$$\frac{r_{d-1} - r_d}{c_d - c_{d-1}} < \tau.$$

The first condition stops when the residual is small enough. The second stops when the next refinement gives too little residual reduction per unit cost.

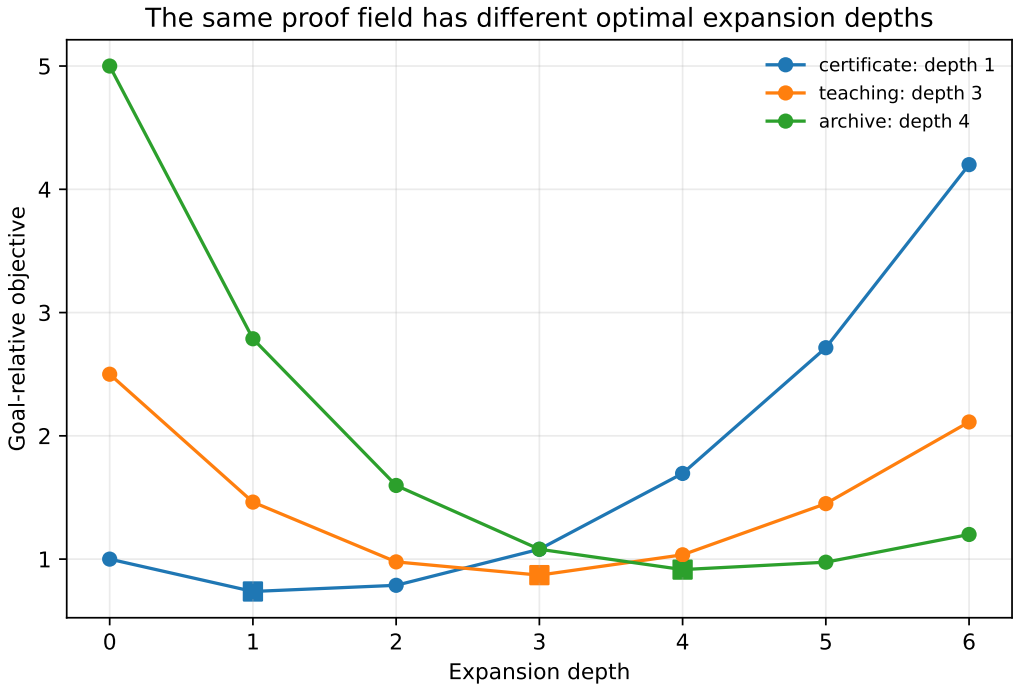


Figure 12: Certificate, teaching, and archive goals select different depths in one finite model.

Deeper is not automatically better. A formal checker may need primitive steps. A specialist paper may cite a standard theorem. A teaching account may expand only the conceptual bottleneck. An archive may retain a failed branch that neither certificate requires.

20 Worked finite examples

The accompanying repository contains three canonical JSON proof fields and several auxiliary models. The examples are small enough to inspect by hand and complete enough to exercise the formal distinctions.

20.1 A fixed AND-DAG

The field `examples/and_dag.json` has required premise A, a definition D, lemmas L_1 and L_2, and target B. It also contains a worked example, a failed route, and a boundary case. The unique verification core is

$$\{A, D, L_1, L_2, B\}.$$

The optional nodes have zero declared verification value for B and positive discovery, meaning, or bridge value. This is the executable instance behind Figure 6.

20.2 Two certified proofs

The field `examples/alternative_proofs.json` contains a short route and a richer explanatory route. The short route has storage cost 1, while the richer route has storage cost 3 and larger meaning and

discovery values. Exact certificate search chooses

$$A \longrightarrow L_{\text{short}} \longrightarrow B$$

when the objective is storage cost alone. A teaching compiler may rationally choose the other route because it uses a different objective.

20.3 Set Cover encoded as certificate selection

The field `examples/set_cover_certificate.json` contains four optional source lemmas and four obligations. Exact search returns the certificate using S_1 and S_3, with total cost 2. The particular optimum is not important. The example demonstrates how local alternative derivations create a global combinatorial selection problem.

20.4 Bridge retention

The bridge experiment uses ten candidate nodes and sixteen weighted future targets. For budgets zero through five, the script computes both the exact optimum and the greedy solution. The generated data are stored in `data/retention_comparison.csv`. On this instance the two curves coincide. The theorem guarantees only the worst-case $(1 - 1/e)$ factor.

20.5 Correlated review and audit capacity

For one hundred equicorrelated reviewers with $\rho = 0.1$,

$$n_{\text{eff}} = \frac{100}{1 + 99(0.1)} \approx 9.17.$$

The audit example begins with backlog 10. Arrivals 4 and capacity 5 clear the queue. Equal rates hold the backlog at 10. Arrivals 6 and capacity 5 increase it by one per cycle.

Table 1: Canonical finite checks in the version 1.0 release.

Object	Computed result	Mathematical role
AND-DAG	unique five-node core	Theorem 5.3
Alternative field	minimum cost 1	exact finite search
Set Cover field	minimum cost 2	Theorem 6.2 encoding
Bridge model	greedy and exact curves	Theorems 16.1–16.2
Review model	n_{eff} formula	Theorem 20.1
Audit model	three queue regimes	Theorem 21.1

21 Reference implementation

The repository implements the finite theory as the Python package `proof_path_theory`. The public modules are organized by mathematical responsibility.

Module	Responsibility
`model.py`	Typed nodes, justifications, evidence records, acyclicity, ancestry, and finite derivability.
`certificate.py`	Certificate validation, unique AND-DAG cores, exact minimum certificate search, and certificate cost.
`compression.py`	Task sufficiency, recoverability preorder, loss reports, compression debt, representation utilities, and Pareto fronts.
`bridges.py`	Reachability, domination-based criticality, exact retention, greedy retention, and finite submodularity checks.
`review.py`	Equicorrelated covariance, effective reviewer count, generalized least squares, and shared-bias mean squared error.
`audit.py`	Audit queue simulation, rate regimes, and explicit priority scores.
`refinement.py`	Goal-relative refinement decisions, optimal depth, and stopping rules.
`versioning.py`	Affected descendants, append-only checks, ledger snapshots, and provenance digests.
`io.py`	Stable JSON import and export for proof fields and certificates.
`cli.py`	Command-line validation, core extraction, minimum certificate search, and certificate audit.

21.1 Command-line interface

The main commands are:

```
proof-path validate examples/and_dag.json
proof-path core examples/and_dag.json --output core.json
proof-path minimum examples/alternative_proofs.json --output minimum.json
proof-path audit examples/and_dag.json core.json
```

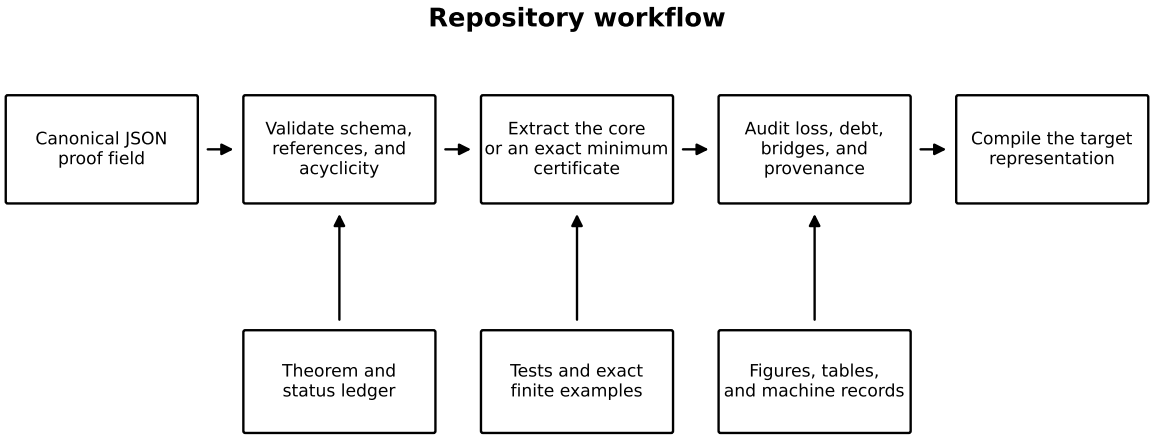


Figure 13: Repository workflow from a canonical JSON field to compiled representations and reproducible records.

21.2 JSON contract

A proof field file contains:

- a schema version and metadata;
- required premises and targets;
- typed nodes with value and cost vectors;
- typed evidence records with scope and certification flags;
- justifications with premise sets, conclusions, rules, evidence references, and certification flags.

The schema is intentionally plain. It can be produced by a proof assistant adapter, a research notebook, a theorem ledger, or a hand-written example.

21.3 Tests and build discipline

Version 1.0 contains sixty-seven automated tests. They cover graph validation, derivability, certificate extraction, alternative proofs, the Set Cover encoding, loss vectors, task sufficiency, bridge submodularity, retention, review covariance, audit queues, refinement, version propagation, JSON round trips, and command-line smoke tests.

A separate deterministic cross-check compares the exact certificate solver with brute-force enumeration over eighteen cost profiles, tests the greedy cardinality guarantee on one hundred generated coverage instances, verifies 1,024 retention-algebra identities, and compares the effective-reviewer formula with direct covariance calculations in 114 parameter settings.

The tests and cross-checks verify the implementation of finite definitions and examples. They do not replace the proofs in this paper. Conversely, the proofs do not guarantee that a software implementation is free of defects. The release keeps both forms of verification.

22 What the theory establishes

The completed finite theory establishes the following structural facts.

1. A proof field can represent conjunction within a justification and alternatives among justifications without confusing candidate edges with certified edges.
2. Derivability in a finite acyclic selected subfield is well defined and computable in topological order.
3. A fixed AND-DAG has a unique minimal verification core.
4. Alternative derivations make minimum certificate selection NP-complete in general.
5. Compression is meaningful only relative to a task family.
6. Task-relative recoverability forms a preorder and a quotient partial order.
7. Verification loss, discovery loss, meaning loss, bridge loss, and audit loss can diverge.
8. Expected reconstruction debt decreases monotonically as more required material is retained.
9. Bridge criticality is domination in the declared future-reach graph.

10. Weighted future reach is monotone and submodular.
11. Greedy cardinality retention has the standard $(1 - 1/e)$ guarantee.
12. Append-only certified updates preserve existing derivability.
13. The forward dependency closure is the smallest universally sound syntactic invalidation set.
14. Review dependence reduces effective independent review, and generalized least squares gives the optimal linear unbiased aggregate under a known covariance model.
15. Constant-rate audit queues have an exact stable, critical, or unstable classification.
16. Optimal proof expansion depth depends on the presentation goal.

These results turn the original methodological insight into a finite mathematical branch with explicit objects, algorithms, complexity, and failure boundaries.

23 Limitations and claim boundaries

Proof Path Theory does not establish a theorem by preserving the history of an argument. A richly documented wrong proof remains wrong. Bridge value is not validity. Explanatory power is not validity. Consensus is not validity. The certified subfield remains the object that carries deductive force.

The dependency graph is a declared model. It records the dependencies that were formalized, not every historical influence on discovery. Tacit knowledge, visual intuition, unrecorded conversations, and changes in vocabulary may be absent.

The value and cost coordinates are task models. Verification can often be specified sharply. Meaning and future bridge value are partly prospective. Their use is justified when assumptions are stated, not when the numbers are treated as intrinsic physical constants.

The finite bridge graph does not predict every future theorem. It supports conditional statements of the form: under these declared future targets and transfer edges, this retention policy preserves this amount of modeled reach.

The exact minimum certificate algorithm is exponential in the worst case. The package makes this limitation explicit through a search-state bound.

The review and queue models are deliberately elementary. They isolate dependence and capacity effects, but they do not model every institutional or psychological feature of mathematical review.

Finally, the paper does not claim that graphs, hypergraphs, proof theory, submodular optimization, covariance-aware aggregation, or queueing recurrences were invented here. The contribution is the formal assembly of these structures around proof compression, discovery fields, mathematical memory, and versioned proof portfolios.

24 Open theorem program

The finite theory leaves several focused directions.

Problem 24.1 (Infinite proof fields)**.** Develop locally finite and countable proof fields with conditions guaranteeing that finite certificates approximate an infinite derivation without losing domain or

convergence information.

Problem 24.2 (Parameterized certificate complexity)**.** Classify minimum certificate selection by treewidth, number of alternative justifications, justification arity, shared-subproof width, and target count. Identify fixed-parameter tractable regimes and approximation bounds.

Problem 24.3 (Probabilistic bridge calibration)**.** Replace declared bridge weights by calibrated posterior models while keeping conjectural reach separate from certified derivability.

Problem 24.4 (Proof assistant interoperability)**.** Construct adapters that export stable claim identifiers, primitive dependencies, scopes, and proof terms from existing formal systems into the common proof-field schema.

Problem 24.5 (Higher proof transformations)**.** Model transformations between proofs, transformations between transformations, and coherent compiler pipelines in a higher categorical setting.

Problem 24.6 (Privacy and attribution)**.** Design proof archives that preserve provenance and priority while allowing private notes, embargoed routes, and sensitive collaboration records to remain access controlled.

Problem 24.7 (Empirical validation)**.** Study real mathematical projects to estimate recovery costs, audit bottlenecks, and the predictive value of bridge-retention scores. Such work should test the models rather than assume them.

Problem 24.8 (Distributed audit systems)**.** Extend the queue model to heterogeneous reviewers, dependent tasks, rework, priority classes, and claims whose audit cost changes after neighboring claims are resolved.

25 Conclusion

A final proof is indispensable, but it is not always the whole mathematical object. The completed theory distinguishes the certificate from the proof field that produced it. It gives that field typed nodes, alternative justifications, evidence records, task-relative compressions, bridge graphs, version histories, and explicit audit models.

The main mathematical boundary is now clear. Verification is a closure property of the certified subfield. Discovery value belongs to a larger graph. Compression can preserve the first while reducing the second. Once that separation is formal, several consequences follow. Minimal certificates can be extracted exactly in fixed AND-DAGs. Alternative derivations create a genuine combinatorial optimization problem. Future-reach retention becomes submodular coverage. Version changes propagate through descendants. Correlated review has an effective sample size. Candidate generation is constrained by audit capacity. Expansion depth depends on purpose.

The final thesis is therefore:

> A proof should be published as a certificate and maintained as a versioned proof field. The certificate preserves deductive validity. The larger field preserves the dependencies, alternatives, examples, failures, provenance, and bridges from which later mathematics may grow.

A Theorem ledger

ID	Result	Status	Executable support
PPT-001	Finite derivability is unique and computable in topological order.	Proved	`ProofField.derive`
PPT-002	An AND-DAG has a unique minimal verification core.	Proved	`unique_and_dag_certificate`
PPT-003	AND-DAG core extraction is linear in relevant ancestry.	Proved	traversal implementation
PPT-004	Minimum Certificate Decision is NP-complete.	Proved	Set Cover example
PPT-005	Certified paths transport functorially.	Proved	structural theorem
PPT-006	Local auditable certificates compose under a certificate functor.	Proved	structural theorem
PPT-007	Task information loss follows from compression collisions.	Proved	task API
PPT-008	Recoverability is a preorder.	Proved	`recoverability_preorder`
PPT-009	The quotient by mutual recoverability is a partial order.	Proved	equivalence API
PPT-010	Minimal sufficient compression depends on the declared task family.	Proved	finite countermodel
PPT-011	Zero verification loss can coexist with positive discovery loss.	Proved	AND-DAG example
PPT-012	Compression debt is monotone under retention.	Proved	`compression_debt`
PPT-013	Bridge criticality equals target domination.	Proved	`dominated_targets`
PPT-014	Weighted future reach is monotone and submodular.	Proved	exhaustive finite checker
PPT-015	Greedy cardinality retention achieves at least $1 - 1/e$.	Proved	greedy and exact solvers
PPT-016	Append-only certified updates preserve derivability.	Proved	versioning check
PPT-017	Forward dependency closure is the minimal closed syntactic review set.	Proved	`affected_descendants`
PPT-018	Equicorrelated review has the stated effective count.	Proved	covariance implementation
PPT-019	GLS weights minimize unbiased linear review variance.	Proved	`generalized_least_squares_weights`
PPT-020	Constant-rate audit queues have exact stable, critical, and unstable regimes.	Proved	queue simulation
PPT-021	Optimal expansion depth depends on the goal.	Proved	refinement implementation
PPT-022	Pure retention projections commute and are idempotent.	Proved	`retention_projection`

ID	Result	Status	Executable support
PPT-023	Retention masks form a Boolean lattice.	Proved	`retention_meet`, `retention_join`

B Reproduction

From the section root:

```
python -m pip install -e . --no-build-isolation
python -m pytest
python scripts/build_examples.py
python scripts/generate_figures.py
proof-path schema proof-field
proof-path validate examples/and_dag.json
proof-path core examples/and_dag.json
proof-path minimum examples/alternative_proofs.json
pdflatex -interaction=nonstopmode -halt-on-error docs/Proof_Path_Theory.tex
pdflatex -interaction=nonstopmode -halt-on-error docs/Proof_Path_Theory.tex
pdflatex -interaction=nonstopmode -halt-on-error docs/Proof_Path_Theory.tex
```

The source archive also contains a wheel build script and a GitHub Actions workflow fragment for a multi-section repository.

C Canonical proof-field schema excerpt

```
{
  "schema_version": "1.0",
  "premises": ["A"],
  "targets": ["B"],
  "nodes": [
    {
      "id": "A",
      "label": "Premise A",
      "kind": "premise",
      "values": {"verification": 1.0},
      "costs": {"storage": 0.2}
    }
  ],
  "evidence": [
    {
      "id": "e-proof",
      "kind": "formal_proof",
      "source": "declared derivation",
      "scope": "",
      "certifies": true
    }
  ],
  "justifications": [
    {
      "id": "j1",
      "premises": ["A"],
      "conclusion": "B",
      "rule": "deduction",
      "certified": true,
      "evidence_ids": ["e-proof"]
    }
  ]
}
```

References

[1] S. R. Buss, editor. *Handbook of Proof Theory*. Elsevier, 1998.

[2] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Second edition, Wiley, 2006.

[3] R. Diestel. *Graph Theory*. Fifth edition, Springer, 2017.

[4] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[5] D. E. Knuth. *Literate Programming*. CSLI Publications, 1992.

[6] S. Mac Lane. *Categories for the Working Mathematician*. Second edition, Springer, 1998.

[7] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions I. *Mathematical Programming*, 14:265–294, 1978.

[8] A. S. Troelstra and H. Schwichtenberg. *Basic Proof Theory*. Second edition, Cambridge University Press, 2000.